

Proximal Policy Optimization

算法详解笔记

千喜 Qx

2026/06/07

GitHub Repository:

github.com/X2024-AI/reinforcement-learning-notes_Qx

Contents

1	背景与动机	1
1.1	从 Policy Gradient 到 PPO	1
1.2	PPO 的设计目标	1
2	标准策略梯度基础	1
2.1	目标函数	1
2.2	策略梯度定理	1
2.3	伪目标函数	2
3	通过重要性采样的离策略学习	2
3.1	离策略学习的必要性	2
3.2	重要性采样恒等式	2
3.3	概率比	2
3.4	保守策略迭代 (CPI) 目标	3
4	破坏性更新的危险	3
4.1	为什么无约束的 L_{CPI} 是危险的	3
4.2	一阶（线性）近似陷阱	3
4.3	TRPO 的解决方案：硬 KL 约束	3
4.4	Fisher Information Matrix 噩梦	4
4.5	PPO 的巧妙解决方案	4
5	PPO 裁剪代理目标	4

5.1	裁剪区间	4
5.2	裁剪代理目标定义	4
5.3	案例分析：裁剪机制如何工作	4
5.3.1	Case 1: 正优势 ($\hat{A}_t > 0$)	5
5.3.2	Case 2: 负优势 ($\hat{A}_t < 0$)	5
5.4	紧急制动机制	5
5.5	数学总结：裁剪如何迫使梯度为零	6
6	完整 PPO 目标函数	6
6.1	带价值函数和熵奖励的完整目标	6
6.2	组件 1: 裁剪代理目标 L_{CLIP}	6
6.3	组件 2: 价值函数损失 L^{VF}	7
6.4	组件 3: 熵奖励 $S[\pi_\theta]$	7
7	优势估计	7
7.1	[Mni+16] 风格：截断 horizon	7
7.2	截断估计器（公式 10）	7
7.3	广义优势估计 (GAE)	8
7.4	从第一性原理推导 GAE	8
7.5	通过 λ 的偏差-方差权衡	9
8	理解 PPO 目标曲线	9
8.1	沿策略更新方向的插值	9
8.2	阅读图表：曲线逐条	9
8.3	为什么红线下降？（“下界”直觉）	10
8.4	终极启示	10
9	PPO 中的 On-Policy 与 Off-Policy	10
9.1	澄清命名法	10
9.2	数学期望约束	11
9.3	为什么标准 PG 在重用数据上失败	11
9.4	PPO 如何克服这个（重要性采样）	11
9.5	解决你关于不准确优势函数的观点	11
10	PPO 算法流程	12
10.1	解码优化指令	12
10.2	实际逐步流程	12
10.3	裁剪如何限制更新的微积分	13
11	完整 PPO 实现	14
11.1	PyTorch 实现与详细注释	14
11.2	超参数含义	18
12	实践项目推荐	19
12.1	项目 1: LunarLander-v3（离散控制）	19
12.2	项目 2: BipedalWalker-v3（连续控制过渡）	19
12.3	项目 3: Atari Pong（基于视觉的 RL）	19
12.4	项目 4: 自定义交易环境	19
13	附录 A: 数学工具	20

13.1	随机梯度下降 (SGD)	20
13.2	<code>nn.Tanh()</code> 激活函数	20
14	附录 B: 符号表	21
15	参考文献	22
16	作者信息	22

1 背景与动机

1.1 从 Policy Gradient 到 PPO

Proximal Policy Optimization (PPO) 代表了强化学习算法的重大进步。要理解其动机，我们必须首先研究现有策略优化方法的格局。

传统的策略梯度方法（如 REINFORCE）虽然简单优雅，但存在一个关键缺陷：它们是 **on-policy** 算法，这意味着每次更新策略参数时都必须丢弃所有收集的数据。这导致极低的样本效率。

另一方面，Trust Region Policy Optimization (TRPO) 引入了一种使用 KL 散度约束策略更新的数学严格方法。然而，TRPO 需要计算 **Fisher Information Matrix** (KL 散度的二阶导数)，对于大型神经网络来说计算量巨大。

PPO 取得了平衡：它保留了 TRPO 约束优化的数学安全性，同时实现了标准策略梯度方法的计算简单性。

1.2 PPO 的设计目标

PPO 的核心创新是**裁剪代理目标函数**。与 TRPO 使用硬 KL 约束不同，PPO 修改目标函数本身来惩罚偏离旧策略过远的策略更新。这种裁剪机制在计算上微不足道，同时有效地防止了破坏性的策略更新。

2 标准策略梯度基础

2.1 目标函数

在强化学习中，我们的根本目标是最大化策略 π_θ 下的期望累计奖励。期望回报定义为：

$$J(\theta) = E_{\tau \sim \pi_\theta}[R(\tau)] \quad (1)$$

其中：

- $\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots)$ ：轨迹（状态和动作的序列）
- $R(\tau) = \sum_{t=0}^T r_t$ ：轨迹的总奖励
- π_θ ：由神经网络权重 θ 参数化的策略

2.2 策略梯度定理

使用似然比技巧，我们可以在不计算环境动力学梯度的情况下推导目标函数的梯度：

$$\nabla_\theta J(\theta) = E_{\tau \sim \pi_\theta} \left[\left(\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right) R(\tau) \right] \quad (2)$$

通过仔细应用**因果原则**（时间 t 之前的奖励不影响时间 t 的梯度），这简化为：

$$\nabla_\theta J(\theta) = \hat{E}_t \left[\nabla_\theta \log \pi_\theta(a_t | s_t) \hat{A}_t \right] \quad (3)$$

其中 $\hat{A}_t$ 是时间步 t 的估计**优势函数**。

符号定义：

- $\hat{E}_t[\cdot]$: 时间步的经验平均（近似期望）
- $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$: 在状态 s_t 下选择动作 a_t 的对数概率梯度
- $\hat{A}_t$: 优势估计，衡量一个动作相比该状态的平均行为好多少

2.3 伪目标函数

相应的伪目标函数（通过梯度上升最大化）为：

$$L_{PG}(\theta) = \hat{E}_t \left[\log \pi_{\theta}(a_t|s_t) \hat{A}_t \right] \quad (4)$$

这是 PPO 构建的基础。

3 通过重要性采样的离策略学习

3.1 离策略学习的必要性

标准策略梯度方法是 **on-policy** 的：每次更新 θ 时，我们必须丢弃采样轨迹并采集新轨迹。这极大地降低了样本效率。

为了提高数据效率，我们希望使用由旧策略 $\pi_{\theta_{old}}$ 收集的数据来优化新策略 π_{θ} 。

3.2 重要性采样恒等式

我们使用**重要性采样**来实现这一点，这是一种将一个分布下的期望重写为另一个分布下期望的数学恒等式：

$$E_{x \sim p}[f(x)] = \int p(x) f(x) dx = \int q(x) \frac{p(x)}{q(x)} f(x) dx = E_{x \sim q} \left[\frac{p(x)}{q(x)} f(x) \right] \quad (5)$$

关键洞察：分布 p 下的期望可以使用来自分布 q 的样本计算，通过重要性权重 $\frac{p(x)}{q(x)}$ 来修正分布不匹配。

3.3 概率比

将此应用于我们的策略目标，我们定义**概率比** $r_t(\theta)$ ：

$$r_t(\theta) = \frac{\pi_{\theta_{old}}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \quad (6)$$

重要观察：如果 $\theta = \theta_{old}$ ，则 $r_t(\theta) = 1$ 。

3.4 保守策略迭代 (CPI) 目标

用这个比率替换标准对数梯度得到保守策略迭代目标：

$$L_{CPI}(\theta) = \hat{E}_t \left[\frac{\pi_{\theta_{old}}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \hat{A}_t \right] = \hat{E}_t \left[r_t(\theta) \hat{A}_t \right] \quad (7)$$

符号定义：

- $\pi_{\theta_{old}}(a_t|s_t)$ ：在状态 s_t 下由旧（行为）策略选择动作 a_t 的概率
- $\pi_{\theta}(a_t|s_t)$ ：新（目标）策略下的概率
- $r_t(\theta)$ ：衡量新策略相比旧策略选择 a_t 的可能性倍数

4 破坏性更新的危险

4.1 为什么无约束的 L_{CPI} 是危险的

无约束地最大化 $L_{CPI}(\theta)$ 是非常危险的。因为 L_{CPI} 只是真目标在 θ_{old} 附近的一阶（线性）近似，巨大的梯度步长可能导致 $r_t(\theta)$ 飙升或暴跌。这迫使策略进入无法恢复的状态，性能崩溃。

4.2 一阶（线性）近似陷阱

在机器学习中，真目标函数（在所有可能的神经网络权重上策略的实际好坏）是一个高度复杂的蜿蜒山脉。

当算法进行一阶近似时，它：

1. 在单一精确坐标 (θ_{old}) 处观察山脉
2. 计算斜率（一阶导数/梯度）
3. 画一条完全直的正切线

局部（小额步长）：直线非常准确。沿直线移动 0.001 个单位，你基本上仍在山上。

全局（巨额步长）：直线完全脱离现实。如果山脉突然变成陡峭的下行悬崖，直线不在乎——它继续指向天空。

对于好动作 ($\hat{A}_t > 0$)，线性近似告诉网络：“你越朝这个方向移动，奖励越高，无限！”优化器盲目信任这条线并迈出巨额步长，迫使 $r_t(\theta)$ 飙升至无穷大。

相反，对于坏动作 ($\hat{A}_t < 0$)，这条线告诉它使概率比降至零，完全破坏策略分布。

4.3 TRPO 的解决方案：硬 KL 约束

TRPO 认识到这种危险，并引入**硬数学约束**使用 KL 散度确保 π_{θ} 保持在 $\pi_{\theta_{old}}$ 附近：

$$\theta_{max} \quad \hat{E}_t \left[r_t(\theta) \hat{A}_t \right] \quad \text{subject to} \quad \hat{E}_t [KL(\pi_{\theta_{old}}(\cdot|s_t) || \pi_{\theta}(\cdot|s_t))] \leq \delta \quad (8)$$

符号定义：

- $KL(\pi_{\theta_{old}} || \pi_{\theta})$ ：衡量策略分布变化的 KL 散度
- δ ：最大允许 KL 散度（“信任区域”半径）

4.4 Fisher Information Matrix 噩梦

为什么 TRPO 计算量巨大：

”信任区域”的形状不是完美的圆——它是一个高度扭曲的多维椭球。在某些权重方向上，微小变化会完全改变策略；在其他方向上，你可以大幅度改变权重而不改变行为。

为了绘制这个安全区域的确切轮廓，TRPO 必须计算 KL 散度梯度如何随权重变化。梯度的导数是二阶导数（称为 **Hessian** 的偏二阶导数矩阵）。

当应用于概率分布时，这个特定的二阶导数矩阵被称为 **Fisher Information Matrix (FIM)**。

计算规模：

假设你的神经网络包含 $P = 1,000,000$ 个参数（权重和偏置）。FIM 是一个大小为 $P \times P$ 的方阵，包含 1 万亿个元素。存储这需要大约 **4 TB 的显存**。

要进行更新步骤，TRPO 必须计算这个矩阵的逆 (F^{-1})，这在计算上是毁灭性的。

4.5 PPO 的巧妙解决方案

PPO 看到了这个计算噩梦，说：”与其花费巨大的计算能力构建一个完美的 4 TB 安全室矩阵 (TRPO)，不如使用我们廉价的线性近似，但添加一个**简单的裁剪函数**来切断试图走太远的梯度。”

5 PPO 裁剪代理目标

5.1 裁剪区间

PPO 不是在策略空间上强制硬约束，而是强制概率比 $r_t(\theta)$ 保持在 1 附近的区间内，通常是：

$$[1 - \epsilon, 1 + \epsilon] \quad (9)$$

其中 $\epsilon \approx 0.2$ 是一个超参数。

5.2 裁剪代理目标定义

PPO 裁剪代理目标定义为：

$$L_{CLIP}(\theta) = \hat{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (10)$$

符号定义：

- $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$ ：将 $r_t(\theta)$ 裁剪到区间 $[1 - \epsilon, 1 + \epsilon]$
- $\min(\cdot, \cdot)$ ：取未裁剪和裁剪目标的最小值

5.3 案例分析：裁剪机制如何工作

为了理解为什么 $\min$ 和 clip 算子有效，我们必须根据动作是否产生正向或负向结果来检查它们的行为。

5.3.1 Case 1: 正优势 ($\hat{A}_t > 0$)

采取的动作获得了超出预期的回报。我们想修改 θ 使这个动作更可能，这将驱动 $r_t(\theta)$ 高于 1。

子情况 1a: 当 $r_t(\theta) \leq 1 + \epsilon$
裁剪函数不触发。目标简化为：

$$L_{CLIP}(\theta) = r_t(\theta)\hat{A}_t \quad (11)$$

梯度是活跃的，网络学习增加这个好动作的概率。

子情况 1b: 当 $r_t(\theta) > 1 + \epsilon$
裁剪项锁定在 $(1 + \epsilon)\hat{A}_t$ 。因为 $\hat{A}_t > 0$ ：

$$r_t(\theta)\hat{A}_t > (1 + \epsilon)\hat{A}_t \quad (12)$$

$\min(\cdot)$ 算子选择较小的值： $(1 + \epsilon)\hat{A}_t$ 。

直觉：一旦策略使动作比之前可能性高出 $1 + \epsilon$ 倍，梯度降至零。这防止算法在单个好轨迹上过度优化。

5.3.2 Case 2: 负优势 ($\hat{A}_t < 0$)

采取的动作获得了低于预期的回报。我们想修改 θ 使这个动作不太可能，这将驱动 $r_t(\theta)$ 低于 1。

子情况 2a: 当 $r_t(\theta) \geq 1 - \epsilon$
裁剪函数不触发。目标是 $r_t(\theta)\hat{A}_t$ 。

子情况 2b: 当 $r_t(\theta) < 1 - \epsilon$

裁剪项锁定在 $(1 - \epsilon)\hat{A}_t$ 。因为 $\hat{A}_t < 0$ ，乘以一个较小的数得到一个较大（较小负）的值：

$$0.5 \times (-2) = -1.0 \quad \text{而} \quad 0.8 \times (-2) = -1.6$$

$\min(-1.0, -1.6)$ 算子选择较小（较负）的值： $(1 - \epsilon)\hat{A}_t$ 。

直觉：当 $r_t(\theta)$ 低于 $1 - \epsilon$ 时，目标变平，意味着智能体不会因为使坏动作更不可能而获得无限赞扬。

5.4 紧急制动机制

如果策略犯了可怕的错误怎么办？

如果动作是坏的 ($\hat{A}_t < 0$) 但策略更新意外地使其更可能 ($r_t(\theta) \gg 1$):

- 裁剪项变为 $(1 + \epsilon)\hat{A}_t$ （一个大负数）
- 未裁剪项 $r_t(\theta)\hat{A}_t$ 变成一个巨大的负数
- $\min(\cdot)$ 算子选择未裁剪的值

这充当未裁剪的紧急制动，允许强大的负梯度将策略有力地拉回。

5.5 数学总结：裁剪如何迫使梯度为零

理解裁剪如何限制更新，我们检查反向传播期间梯度发生了什么。

回想核心优化更新步骤：

$$\theta_{new} = \theta_{old} + \eta \nabla_{\theta} L_{CLIP}(\theta) \quad (13)$$

如果 $\nabla_{\theta} L_{CLIP}(\theta) = 0$ ，则 $\theta_{new} = \theta_{old}$ ，意味着网络完全停止学习该数据点。

当 $r_t(\theta) > 1 + \epsilon$ 时：

$$L_{CLIP}(\theta) = (1 + \epsilon) \hat{A}_t \quad (14)$$

仔细看这个项。 ϵ 是一个常数（如 0.2）， $\hat{A}_t$ 已在数据收集期间计算和冻结。这个项中没有 θ 残留。它是一个平的常数。

因此：

$$\nabla_{\theta} L_{CLIP}(\theta) = \nabla_{\theta} [\text{constant}] = 0 \quad (15)$$

总结：裁剪机制不是物理上阻塞权重；它使损失景观变平。当策略偏离太远时，通过将目标函数变成平坦高原，它剥夺了优化器的梯度。没有梯度，优化器无法计算移动方向，有效地锁定策略。

6 完整 PPO 目标函数

6.1 带价值函数和熵奖励的完整目标

当使用深度神经网络实现 PPO 时，策略网络（Actor）和价值网络（Critic）通常共享早期层。为了同时优化两者同时防止提前收敛，最终 PPO 损失函数结合了三个组件：

$$L^{CLIP+VF+S}(\theta) = \hat{E}_t [L_{CLIP}^t(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_{\theta}](s_t)] \quad (16)$$

其中：

$$L_t^{VF}(\theta) = (V_{\theta}(s_t) - V_{targ}^t)^2 \quad (17)$$

符号定义：

- $L_{CLIP}^t(\theta)$ ：时间 t 的裁剪代理目标
- $L_t^{VF}(\theta)$ ：价值函数损失——预测值 $V_{\theta}(s_t)$ 与目标值 V_{targ}^t 之间的均方误差
- $S[\pi_{\theta}](s_t)$ ：状态 s_t 处策略熵，衡量策略的随机性
- c_1 ：价值函数损失系数（通常为 0.5）
- c_2 ：熵奖励系数（通常为 0.01）

6.2 组件 1：裁剪代理目标 L_{CLIP}

如第 5 章所推导的，这是防止破坏性策略更新的核心 PPO 目标：

$$L_{CLIP}(\theta) = \hat{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right] \quad (18)$$

6.3 组件 2：价值函数损失 L^{VF}

价值函数损失迫使 Critic 准确预测状态值：

$$L_t^{VF}(\theta) = (V_\theta(s_t) - V_{targ}^t)^2 = (V_\theta(s_t) - \hat{R}_t)^2 \quad (19)$$

重要性：

- Critic 网络 $V_\theta(s)$ 估计从状态 s 开始的期望累计奖励
- V_{targ}^t 是目标值（通常使用 GAE 或 n 步回报计算）
- 最小化这个 MSE 损失提高优势估计的准确性
- 更好的优势估计带来更稳定和高效的学习

6.4 组件 3：熵奖励 $S[\pi_\theta]$

熵衡量策略分布的**随机性**或**不确定性**：

$$S[\pi_\theta](s) = - \sum_a \pi_\theta(a|s) \log \pi_\theta(a|s) \quad (20)$$

对于具有 softmax 策略的离散动作空间，这是动作概率分布的香农熵。

熵奖励的作用：

- **鼓励探索：**确定性策略（所有概率在一个动作上）的熵为零。添加熵奖励防止策略过早崩溃
- **防止提前收敛：**没有熵奖励，策略可能过早锁定在次优确定性策略上
- **系数 c_2 ：**控制探索强度。太高 = 随机行为；太低 = 提前收敛

重要说明：熵奖励在损失函数中被减去，意味着我们**最大化**熵同时最大化裁剪目标。这通过添加 $-c_2 \times S$ （负熵）到损失来实现，等价于最小化 $c_2 \times (-S)$ 。

7 优势估计

7.1 [Mni+16] 风格：截断 horizon

一种策略梯度实现风格，在 [Mni+16] (Mnih et al. 2016 A3C 论文) 中流行，非常适合与**循环神经网络**一起使用，运行策略 T 个时间步（其中 T 远小于回合长度），并使用收集的样本进行更新。

这种风格需要不超出时间步 T 的优势估计器。

7.2 截断估计器（公式 10）

[Mni+16] 使用的估计器是：

$$\hat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T) \quad (21)$$

其中 t 指定在给定长度- T 轨迹段内的时间索引 $[0, T]$ 。

符号定义：

- $V(s_t)$: 截断段开始处的价值估计
- $r_t, r_{t+1}, \dots, r_{T-1}$: 截断窗口内的观测奖励
- $\gamma^{T-t}V(s_T)$: 截断段末尾的 bootstrap 值
- $V(s_t)$ 前的负号修正起始值 (基线减法)

直觉: 这个估计器正好向前看 $T-t$ 步 (加上末尾的 bootstrap 值), 使其适用于 RNN 中截断 BPTT (通过时间的反向传播)。

7.3 广义优势估计 (GAE)

推广截断选择, 我们可以使用**截断广义优势估计**版本, 当 $\lambda = 1$ 时简化为公式 (10):

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots + (\gamma\lambda)^{T-t+1}\delta_{T-1} \quad (22)$$

其中 **TD 误差** δ_t 定义为:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (23)$$

符号定义:

- δ_t : 时间 t 的 TD 误差 (价值估计偏离了多少)
- γ : 折扣因子
- λ : GAE 平滑参数 (平衡偏差和方差)
- $V(s_{t+1})$: 下一状态的价值估计

7.4 从第一性原理推导 GAE

GAE 公式表示 n 步 TD 误差的指数加权移动平均:

$$\hat{A}_t = \sum_{n=0}^{T-t-1} (\gamma\lambda)^n \delta_{t+n} \quad (24)$$

让我们验证当 $\lambda = 1$ 时这简化为公式 (10):

$$\hat{A}_t = \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} + \dots + \gamma^{T-t-1}\delta_{T-1} \quad (25)$$

展开每个 δ :

$$\begin{aligned} \delta_t &= r_t + \gamma V(s_{t+1}) - V(s_t) \\ \delta_{t+1} &= r_{t+1} + \gamma V(s_{t+2}) - V(s_{t+1}) \\ &\vdots \end{aligned}$$

经过代数消去的望远镜求和后, 这简化为正是公式 (10)。

7.5 通过 λ 的偏差-方差权衡

GAE_LAMBDA (λ) 是平衡两个 evil 的调谐旋钮：

极端 1: $\lambda = 0$ (高偏差, 低方差)

- 算法只向前看一步
- 严重依赖 Critic 网络的价值估计
- 训练初期, 当 Critic 无知时, 这导致系统性偏差

极端 2: $\lambda = 1$ (低偏差, 高方差)

- 算法一直看到回合结束 (蒙特卡洛回报)
- 不依赖价值估计 (无偏)
- 然而, 随机环境导致方差在长 horizon 上指数级复合

最佳点: $\lambda = 0.95$

通过设置 GAE_LAMBDA = 0.95, 我们取所有多步向前看的指数加权平均:

- 严重倾向于向前看远方 (低偏差)
- 应用温和的折扣因子来抑制后期步骤的混乱噪声 (可控方差)

数十年的经验 RL 研究表明, ****0.95**** 是平衡这种权衡的几乎通用最优选择。

8 理解 PPO 目标曲线

8.1 沿策略更新方向的插值

神经网络权重生活在具有数千或数百万维的空间中, 使可视化整个损失景观成为不可能。为了解决这个问题, 研究人员通过该巨大空间取一个单一的 **1D ”切片”**。

设 θ_{old} 为起始权重, θ_{new} 为 PPO 提出的目标权重。”方向”是连接它们的直线:

$$\theta(\alpha) = \theta_{old} + \alpha(\theta_{new} - \theta_{old}) \quad (26)$$

阅读 x 轴 (线性插值因子 α):

- 在 $\alpha = 0$: 恰好站在 θ_{old} (在进行任何更新之前)
- 在 $\alpha = 1$: 恰好站在 θ_{new} (PPO 决定采取的实际步长)
- 在 $\alpha > 1$: 故意超调看看会发生什么

8.2 阅读图表: 曲线逐条

该图显示当我们将 α 上下调节时, 四个不同数学指标如何变化。

橙线 (L_{CPI}): 危险的乐观主义者

这是标准的未裁剪策略梯度目标 ($r_t \hat{A}_t$)。注意当我们推过 $\alpha = 1$ 时, 橙线继续飙升向上。

问题：如果你使用标准策略梯度，优化器会说”哇，向右走产生巨大奖励！让我们大步迈向 $\alpha = 5$ 或 $\alpha = 10$ ！”这导致破坏性策略更新。

蓝线 ($\hat{E}_t[KL_t]$): 隐藏的悬崖

这追踪 KL 散度（策略实际行为变化了多少）。注意它在 $\alpha = 0$ 处是平的，但随着 α 增加曲线剧烈向上。

问题：这显示了橙线盲目看不到的危险。如果优化器盲目跟随橙线向右走，策略变化如此剧烈以至于性能将完全崩溃。

绿线（仅裁剪项）

这显示隔离裁剪 piece 的行为： $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)\hat{A}_t$ 。当你向右走时，它完全变平，一旦走太远就剥夺优化器的任何梯度。

红线 (L_{CLIP}): PPO 的杰作

这是实际的 PPO 目标函数，取橙线和绿线的最小值。注意其美丽的形状：

- 在 0 和 1 之间，它向上爬升，允许模型高效学习
- 它几乎恰好在 $\alpha = 1$ 处达到峰值
- 一旦你超调超过 1，红线开始迅速下降

8.3 为什么红线下降？（”下界”直觉）

PPO 的 L_{CLIP} 是 L_{CPI} 的下界，对过大的策略更新进行惩罚。

为什么它下降而不是保持平坦？

PPO 的一次迭代计算整个批次数据上的平均（期望 $\hat{E}_t$ ），包含数百个不同的状态-动作对。

对于好动作 ($\hat{A}_t > 0$)，超调使它们变平（跟随绿线）。

但对于坏动作 ($\hat{A}_t < 0$)，过大步长可能意外地使那些坏动作更可能。对于那些特定的坏转换，未裁剪项 ($r_t \hat{A}_t$) 变成一个巨大的负数。

因为 PPO 公式使用 $\min(\cdot)$ 算子，它迫使目标忽略好动作的平坦绿线，而是关注坏动作的巨大负惩罚。这将整个平均（红线）向下拖。

8.4 终极启示

PPO 的数学有效地重塑了损失景观。它将危险的盲目陡峭山峰（橙线）变成安全、定义良好的峰值（红线），迫使你的深度学习优化器恰好在完美步长 ($\alpha = 1$) 处停止，此时更新是安全的。

9 PPO 中的 On-Policy 与 Off-Policy

9.1 澄清命名法

严格来说，PPO 是一个 on-policy 算法。在 RL 中，如果算法可以将经验存储在回放缓冲区中并重用由数百上千步前的旧策略收集的数据，则是 off-policy（如 DQN 或 SAC）。PPO 不能做到这一点。如果你尝试用完全不同训练会话的数据喂养 PPO，算法会失败。

然而，PPO 通常被描述为具有 off-policy 特性，因为它是局部 off-policy。它使用一个策略收集一批数据，冻结该策略，然后重用相同的数据更新权重 K 个 epoch。

9.2 数学期望约束

你关于“更新后不准确”的直觉非常接近真相，但这不是关于总奖励 (G_t) 或优势 ($\hat{A}_t$) 的准确性。这是关于概率的基本规则：**** 数学期望约束 ****。

仔细看标准策略梯度目标函数：

$$L_{PG}(\theta) = E_{(s_t, a_t) \sim \pi_\theta} [\log \pi_\theta(a_t | s_t) \hat{A}_t] \quad (27)$$

注意期望下的下标： $(s_t, a_t) \sim \pi_\theta$ 。这意味着这个方程**数学上仅在数据池中的状态和动作直接来自你当前优化的确切策略 π_θ 时才有效**。

9.3 为什么标准 PG 在重用数据上失败

分解如下：

1. 你使用当前策略权重 θ_0 收集数据
2. 你运行一步梯度上升。你的权重从 $\theta_0 \rightarrow \theta_1$ 变化
3. 你现在有一个新策略 π_{θ_1}
4. 如果你尝试使用相同的数据进行第二步梯度下降，你正在用来自 π_{θ_0} 的数据评估 $L_{PG}(\theta_1)$
5. 因为 $\pi_{\theta_1} \neq \pi_{\theta_0}$ ，**数学期望崩溃**。梯度指向根本错误的方向，导致策略破坏性地过拟合于它不再代表的分布

9.4 PPO 如何克服这个（重要性采样）

PPO 使用**重要性采样**绕过这个期望约束。这是一种数学恒等式，允许你使用从分布 Q 采样的数据计算分布 P 下的期望。

PPO 不是优化标准对数损失，而是优化一个比率：

$$r_t(\theta) = \frac{\pi_{\theta_{old}}(a_t | s_t)}{\pi_\theta(a_t | s_t)} \quad (28)$$

目标函数变为：

$$L_{CPI}(\theta) = E_{(s_t, a_t) \sim \pi_{\theta_{old}}} \left[\frac{\pi_{\theta_{old}}(a_t | s_t)}{\pi_\theta(a_t | s_t)} \hat{A}_t \right] \quad (29)$$

注意现在的下标：数据允许来自 $\pi_{\theta_{old}}$ 。

当 PPO 运行 K 个 epoch 时， $\pi_{\theta_{old}}$ 保持完全冻结。当 θ 在那些 K 个 epoch 上更新和变化时，比率 $r_t(\theta)$ **数学上修正了数据略微陈旧的事实**。它缩放梯度上下以补偿分布偏移。

9.5 解决你关于不准确优势函数的观点

统计噪声（不准确的优势函数）和**系统性数学错误**（违反期望约束）之间有巨大差异。

1. **训练开始时的不准确优势（统计噪声）**：Critic 网络是新而且预测能力差，所以 $\hat{A}_t$ 高度不准确和有噪声。然而，在深度学习中，神经网络非常擅长过滤零均值随机噪声如果你喂养它们足够多的数据。只要梯度平均指向正确方向，模型将学习

2. **违反期望约束（系统性错误）**：如果你在不进行重要性采样的情况下在标准策略梯度中重用数据，你不仅仅是处理有噪声的数据——你正在给优化器喂一个**数学上欺诈的梯度**。优化器认为它正在计算当前策略景观的斜率，但实际上它正在计算来自旧策略的扭曲斜率。这种系统性偏差迅速将策略网络推下悬崖

10 PPO 算法流程

10.1 解码优化指令

短语“**Optimize surrogate L wrt θ , with K epochs and minibatch size $M \leq NT$** ”只是数据回收管道的紧凑机器学习速记。

Table 1: PPO 变量定义

变量	是什么	实际含义
N	并行环境数	同时运行以收集数据的游戏/模拟实例数
T	时间步数	每个环境在暂停更新网络前收集的体验步数
NT	总数据池	收集的完整体验批次（ $N \times T$ 总转换数）
M	小批次大小	从 NT 采样用于计算单次梯度步长的较小数据块
K	Epoch 数	在丢弃之前算法循环遍历整个 NT 数据池的次数

10.2 实际逐步流程

与标准策略梯度不同（收集数据、更新一次、立即丢弃），PPO 执行以下操作：

步骤 1：收集数据

- 智能体在 N 个环境中播放 T 步
- 累积 NT 个总数据点的大池：状态、动作、奖励、价值、对数概率、done 标志

步骤 2：冻结旧策略

- 将当前策略权重保存为 $\pi_{\theta_{old}}$
- 这用于计算比率 $r_t(\theta)$ 的分母

步骤 3：Epoch 循环（运行 K 次）

对于每个 epoch：

1. 洗牌整个 NT 数据点池
2. 将数据池切分成大小为 M 的较小小批次

3. 对于每个小批次：

- 计算 PPO 损失 (L_{CLIP})
- 使用优化器（如 Adam 或 SGD）计算梯度并更新实际网络权重 θ

步骤 4：丢弃并重复

- 在对数据进行 K 次完整遍历后，丢弃 NT 数据
- 用更新后的策略从步骤 1 重复

10.3 裁剪如何限制更新的微积分

理解裁剪如何精确限制更新，我们检查反向传播期间梯度发生了什么。

回想核心优化更新步骤：

$$\theta_{new} = \theta_{old} + \eta \nabla_{\theta} L_{CLIP}(\theta) \quad (30)$$

如果 $\nabla_{\theta} L_{CLIP}(\theta) = 0$ ，则 $\theta_{new} = \theta_{old}$ ，意味着网络完全停止学习该数据点。

区域 1：正常区域（无裁剪）

当新策略与旧策略相比没有太大变化时，比率接近 1 ($1 - \epsilon < r_t(\theta) < 1 + \epsilon$)。min 算子选择未裁剪目标：

$$L_{CLIP}(\theta) = r_t(\theta) \hat{A}_t = \frac{\pi_{\theta_{old}}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \hat{A}_t \quad (31)$$

当我们取关于网络权重 θ 的导数时，梯度是**活跃的**并驱动权重更新。

区域 2：裁剪区域（更新”制动”）

现在想象网络更新其权重，在下一个 epoch 中遇到完全相同的数据点。策略变化如此之大，以至于现在非常可能选择那个动作。比率超过阈值： $r_t(\theta) > 1 + \epsilon$ 。

此时，min 算子触发并迫使目标函数选择裁剪版本：

$$L_{CLIP}(\theta) = (1 + \epsilon) \hat{A}_t \quad (32)$$

仔细看这个项。 ϵ 是一个常数（如 0.2）， $\hat{A}_t$ 已在数据收集期间计算和冻结。**这个项中没有 θ 残留**。它是一个平的常数。

当我们尝试计算梯度时：

$$\nabla_{\theta} L_{CLIP}(\theta) = \nabla_{\theta} [\text{constant}] = 0 \quad (33)$$

因为梯度降至**恰好零**，SGD 更新公式计算为：

$$\theta \leftarrow \theta + \eta(0) = \theta$$

网络**停止学习**该数据点。

11 完整 PPO 实现

11.1 PyTorch 实现与详细注释

```

1 import gymnasium as gym
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from torch.distributions.categorical import Categorical
7
8 # Hyperparameters
9 ENV_NAME = "CartPole-v1"
10 LEARNING_RATE = 3e-4
11 TOTAL_TIMESTEPS = 100000
12 TRAIN_COEFFICIENTS = {"value": 0.5, "entropy": 0.01}
13
14 # PPO Specific Hyperparameters
15 ROLLOUT_STEPS = 2048      # N * T (Total data pool size per
    iteration)
16 MINIBATCH_SIZE = 64      # M
17 PPO_EPOCHS = 10          # K
18 CLIP_EPSILON = 0.2       # epsilon
19 GAMMA = 0.99             # Discount factor
20 GAE_LAMBDA = 0.95        # Smoothing factor for GAE
21
22 class ActorNetwork(nn.Module):
23     def __init__(self, state_dim, action_dim):
24         super(ActorNetwork, self).__init__()
25         self.network = nn.Sequential(
26             nn.Linear(state_dim, 64),
27             nn.Tanh(),
28             nn.Linear(64, 64),
29             nn.Tanh(),
30             nn.Linear(64, action_dim)
31         )
32
33     def get_action_distribution(self, state):
34         logits = self.network(state)
35         return Categorical(logits=logits)
36
37 class CriticNetwork(nn.Module):
38     def __init__(self, state_dim):
39         super(CriticNetwork, self).__init__()
40         self.network = nn.Sequential(
41             nn.Linear(state_dim, 64),
42             nn.Tanh(),
43             nn.Linear(64, 64),
44             nn.Tanh(),
45             nn.Linear(64, 1)
46         )

```

```

47
48     def forward(self, state):
49         return self.network(state)
50
51 class PPOAgent:
52     def __init__(self, state_dim, action_dim):
53         self.actor = ActorNetwork(state_dim, action_dim)
54         self.critic = CriticNetwork(state_dim)
55
56         self.actor_optimizer = optim.Adam(self.actor.parameters()
57                                             , lr=LEARNING_RATE)
58         self.critic_optimizer = optim.Adam(self.critic.parameters
59                                             (), lr=LEARNING_RATE)
60
61     def train_step(self, states, actions, old_log_probs, returns,
62                   advantages):
63         # Convert data to PyTorch tensors
64         states = torch.tensor(np.array(states), dtype=torch.
65                               float32)
66         actions = torch.tensor(np.array(actions), dtype=torch.
67                                long)
68         old_log_probs = torch.tensor(np.array(old_log_probs),
69                                      dtype=torch.float32)
70         returns = torch.tensor(np.array(returns), dtype=torch.
71                                float32).unsqueeze(1)
72         advantages = torch.tensor(np.array(advantages), dtype=
73                                   torch.float32)
74
75         # Normalize advantages for training stability
76         advantages = (advantages - advantages.mean()) / (
77                       advantages.std() + 1e-8)
78
79         dataset_size = len(states)
80
81         # K Epochs Loop
82         for _ in range(PPO_EPOCHS):
83             permutation = torch.randperm(dataset_size)
84
85             # Minibatch Loop
86             for start_idx in range(0, dataset_size,
87                                   MINIBATCH_SIZE):
88                 batch_indices = permutation[start_idx:start_idx +
89                                           MINIBATCH_SIZE]
90
91                 b_states = states[batch_indices]
92                 b_actions = actions[batch_indices]
93                 b_old_log_probs = old_log_probs[batch_indices]
94                 b_returns = returns[batch_indices]
95                 b_advantages = advantages[batch_indices]
96
97                 # Actor Loss Calculation

```

```

87         dist = self.actor.get_action_distribution(
88             b_states)
89         new_log_probs = dist.log_prob(b_actions)
90         entropy = dist.entropy().mean()
91
92         # Probability Ratio  $r_t(\theta)$ 
93         ratios = torch.exp(new_log_probs -
94                             b_old_log_probs)
95
96         # Clipped Surrogate Objective
97         surr1 = ratios * b_advantages
98         surr2 = torch.clamp(ratios, 1.0 - CLIP_EPSILON,
99                             1.0 + CLIP_EPSILON) * b_advantages
100         actor_loss = -torch.min(surr1, surr2).mean()
101
102         # Critic Loss Calculation (MSE)
103         state_values = self.critic(b_states)
104         critic_loss = nn.MSELoss()(state_values,
105                                     b_returns)
106
107         # Total Objective Optimization
108         self.actor_optimizer.zero_grad()
109         actor_total_loss = actor_loss -
110             TRAIN_COEFFICIENTS["entropy"] * entropy
111         actor_total_loss.backward()
112         self.actor_optimizer.step()
113
114         self.critic_optimizer.zero_grad()
115         critic_total_loss = TRAIN_COEFFICIENTS["value"] *
116             critic_loss
117         critic_total_loss.backward()
118         self.critic_optimizer.step()
119
120     def main():
121         env = gym.make(ENV_NAME)
122         state_dim = env.observation_space.shape[0]
123         action_dim = env.action_space.n
124
125         agent = PPOAgent(state_dim, action_dim)
126
127         state, _ = env.reset()
128         global_step = 0
129         episode_rewards = []
130         current_episode_reward = 0
131
132         while global_step < TOTAL_TIMESTEPS:
133             # 1. Storage buffers for Rollout Data Pool (N * T)
134             b_states, b_actions, b_log_probs, b_rewards, b_dones,
135                 b_values = [], [], [], [], [], []
136
137             # 2. Data Collection Phase

```

```

131     for _ in range(ROLLOUT_STEPS):
132         global_step += 1
133         state_t = torch.tensor(state, dtype=torch.float32)
134
135         with torch.no_grad():
136             dist = agent.actor.get_action_distribution(
137                 state_t)
138             action = dist.sample().item()
139             log_prob = dist.log_prob(torch.tensor(action)).
140                 item()
141             value = agent.critic(state_t).item()
142
143         next_state, reward, terminated, truncated, _ = env.
144             step(action)
145         done = terminated or truncated
146         current_episode_reward += reward
147
148         b_states.append(state)
149         b_actions.append(action)
150         b_log_probs.append(log_prob)
151         b_rewards.append(reward)
152         b_dones.append(done)
153         b_values.append(value)
154
155         state = next_state
156         if done:
157             state, _ = env.reset()
158             episode_rewards.append(current_episode_reward)
159             if len(episode_rewards) % 10 == 0:
160                 print(f"Step: {global_step} | Avg Reward (
161                     Last 10 Eps): {np.mean(episode_rewards
162                         [-10:]).2f}")
163             current_episode_reward = 0
164
165     # Compute bootstrap value for the final unfinished step
166     with torch.no_grad():
167         next_value = agent.critic(torch.tensor(state, dtype=
168             torch.float32)).item() if not done else 0
169
170     # 3. Generalized Advantage Estimation (GAE) Calculation
171     b_advantages = np.zeros_like(b_rewards, dtype=np.float32)
172     b_returns = np.zeros_like(b_rewards, dtype=np.float32)
173     last_gae_lam = 0
174
175     for t in reversed(range(ROLLOUT_STEPS)):
176         next_non_terminal = 1.0 - b_dones[t]
177         next_val = b_values[t + 1] if t < ROLLOUT_STEPS - 1
178             else next_value
179
180     # Delta (temporal difference error)
181     delta = b_rewards[t] + GAMMA * next_val *

```

```

        next_non_terminal - b_values[t]
175
        # GAE tracking formula
176        b_advantages[t] = last_gae_lam = delta + GAMMA *
177            GAE_LAMBDA * next_non_terminal * last_gae_lam
178        b_returns[t] = b_advantages[t] + b_values[t]
179
180        # 4. Policy Optimization Phase
181        agent.train_step(b_states, b_actions, b_log_probs,
182            b_returns, b_advantages)
183
184    env.close()
185
186    if __name__ == "__main__":
187        main()

```

Listing 1: PPO 算法 PyTorch 实现

Table 2: 代码核心要点解读

代码段	数学含义
Categorical(logits).sample()	从 softmax 策略分布 $\pi_{\theta}(a s)$ 采样动作
torch.clamp(ratios, 1-CLIP_EPSILON, 1+CLIP_EPSILON)	实现 $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$
torch.min(surr1, surr2)	实现 L_{CLIP} 中的 $\min(\cdot, \cdot)$ 算子
-torch.min(surr1, surr2).mean()	负号因为我们执行梯度下降（最小化负 PPO 损失 = 最大化 L_{CLIP} ）
GAE tracking formula	计算 $A_t = \delta_t + \gamma \lambda \delta_{t+1} + \dots$
b_returns[t] = b_advantages[t] + b_values[t]	计算目标值 $V_{targ}^t = A_t + V(s_t)$

11.2 超参数含义

Table 3: 超参数表

超参数	值	含义
CLIP_EPSILON	0.2	控制策略在一次更新中可以偏离 $\pi_{\theta_{old}}$ 的程度

超参数	值	含义
GAE_LAMBDA	0.95	平衡优势估计中的偏差-方差权衡
ROLLOUT_STEPS	2048	每次更新前收集的总样本数 ($N \times T$)
PPO_EPOCHS	10	重用相同数据更新 θ 的次数
MINIBATCH_SIZE	64	每个小批次的梯度计算大小
GAMMA	0.99	未来奖励的折扣因子
LEARNING_RATE	3e-4	Adam 优化器的步长

12 实践项目推荐

12.1 项目 1: LunarLander-v3 (离散控制)

这是什么：一个经典的向量环境，你必须使用主推进器和侧推进器安全地将月球着陆器模块引导到着陆垫上。

为什么这是一个很好的测试：它引入了高度不宽容的物理。如果你的推进器发射太早或太晚，你就会坠毁。这是一个测试你的价值网络基线准确性的优秀环境，因为它包含一个明显的局部最小值（永远悬停恐惧着陆）。

12.2 项目 2: BipedalWalker-v3 (连续控制过渡)

这是什么：一个双足机器人必须学会在不平坦的地形上向前行走而不会摔倒的 2D 环境。

为什么这是一个很好的测试：这需要修改你的 Actor 网络来处理连续动作空间（输出高斯分布的均值 μ 和方差 σ^2 而不是离散 logits）。PPO 在连续机器人任务中大放异彩；看着你的步行者从不协调的甩动变成稳定的冲刺是非常有成就感的。

12.3 项目 3: Atari Pong (基于视觉的 RL)

这是什么：使用如 gym-super-mario-bros 或 stable-baselines3 的 Atari 环境包装经典视频游戏模拟器。

为什么这是一个很好的测试：它迫使你用卷积神经网络 (CNN) 替换代码中的简单多层感知器 (MLP)。你的智能体必须学会读取原始像素数组，提取空间信息，并直接映射到游戏控制。

12.4 项目 4: 自定义交易环境

这是什么：编写一个读取历史股票或加密货币价格数据的自定义 Gymnasium 子类。动作是 [0: Hold, 1: Buy, 2: Sell]，奖励是你的投资组合净值的 $\mathbb{E}$ 化。

为什么这是一个很好的测试：这将理论桥接到现实世界的应用。PPO 被量化交易公司广泛使用，因为其裁剪目标防止了当金融市场经历突然的波动性变化时灾难性的、不稳定的交易调整。

13 附录 A：数学工具

13.1 随机梯度下降 (SGD)

数学公式：

在标准梯度下降中，我们通过计算整个数据集损失函数 $J(\theta)$ 的梯度来更新模型参数 θ 。

在随机梯度下降中，我们通过在每个迭代中仅计算一个随机选择的训练样本（或单个迷你批次）的损失来近似那个真梯度。

SGD 在步骤 t 的数学更新规则是：

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L_i(\theta_t) \quad (34)$$

符号定义：

- θ_t ：步骤 t 时的当前模型参数向量
- θ_{t+1} ：下一步的更新参数
- η (Eta)：学习率——决定我们沿梯度下行的步长有多大
- i ：从训练数据集中随机选择的索引
- $L_i(\theta_t)$ ：仅在训练实例 (x_i, y_i) 上评估的损失函数
- $\nabla_{\theta} L_i(\theta_t)$ ：损失关于 θ 的梯度（偏导数向量）

SGD 的权衡：

Table 4: SGD 优缺点

优点	缺点
计算廉价：计算 1 个样本的梯度需要大大减少内存和时间	噪声路径：参数更新有高方差
更快收敛：因为它不断更新参数	永不定居：由于噪声，它在全局最小值周围振荡而不是精确停在那里
在线学习：可以在新数据流入时即时更新模型而不从头再训练	

13.2 nn.Tanh() 激活函数

什么是 nn.Tanh() ?

nn.Tanh() 代表双曲正切激活函数。它将任何输入数字压缩到 -1.0 到 1.0 之间的严格、可预测范围：

$$\tanh(x) = \frac{e^x + e^{-x}}{e^x - e^{-x}} \quad (35)$$

为什么在 PPO 中使用 Tanh 而不是 ReLU ?

虽然 `nn.ReLU()` (将所有负数变为 0) 是计算机视觉之王, 但 Tanh 在强化学习策略中更受欢迎:

1. **零中心输出:** 它将负输入映射到负输出, 正输入映射到正输出, 以零为中心其行为。这使得在上下移动概率时梯度更新更干净、更平衡
2. **更平滑分布:** Tanh 是一条光滑、连续的曲线。因为 Actor 网络输出概率 (logits) 形成策略分布, 光滑曲线防止动作概率中突然、刺耳的尖峰

14 附录 B: 符号表

Table 5: 符号表

符号	含义
$\pi_{\theta}(a s)$	参数化策略——在状态 s 下选择动作 a 的概率
θ	策略网络参数 (权重和偏置)
θ_{old}	前一次迭代的策略参数 (行为策略)
$J(\theta)$	期望回报目标函数
$r_t(\theta)$	概率比: $\frac{\pi_{\theta_{old}}(a_t s_t)}{\pi_{\theta}(a_t s_t)}$
L_{CPI}	保守策略迭代目标
L_{CLIP}	PPO 裁剪代理目标
L^{VF}	价值函数损失: $(V_{\theta}(s_t) - V_{targ}^t)^2$
$S[\pi_{\theta}]$	状态 s 处策略分布的熵
$\hat{A}_t$	时间 t 的估计优势
δ_t	TD 误差: $r_t + \gamma V(s_{t+1}) - V(s_t)$
γ	折扣因子
λ	
(GAE_LAMBDA)	GAE 平滑参数
ϵ	
(CLIP_EPSILON)	裁剪区间半宽 (通常为 0.2)
c_1	价值函数损失系数
c_2	熵奖励系数
N	并行环境数
T	每次 rollout 的时间步数
M	小批次大小
K	Epoch 数 (数据遍历次数)
∇_{θ}	关于 θ 的梯度
$\hat{E}_t[\cdot]$	时间步的经验平均
$KL(p q)$	从分布 p 到 q 的 KL 散度
F	Fisher Information Matrix

15 参考文献

- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.
- Mnih, V., Badia, A. P., Mirza, M., et al. (2016). Asynchronous Methods for Deep Reinforcement Learning. *International Conference on Machine Learning*.

16 作者信息

作者信息

基本信息：杨骞玺 - 06 年 - 北京理工大学 - 机器人工程专业

自我介绍：一位刚刚入门具身科研领域的学徒，未来想长期从事具身方向的科研和创业，用机器人替代人类重复的体力劳动。如果你对具身智能感兴趣，或是也在做自己喜欢的事，或是和我一样都喜欢阅读等等，都欢迎大家加我 vx 交朋友，我很丰富，无法简介；)

vx： Y1832800
